AWS VPC PEERING

PROJECT REPORT

Student Name:Kanchana Kodag

PRN:23SC114501092

University:Sanjay Ghodawat University Kolhapur

Internship/Batch:EWB AWS Batch

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Submission Date:31/10/2026

1. ABSTRACT

This project demonstrates private network connectivity between two Amazon Virtual Private Clouds (VPCs) using VPC Peering.

VPC A contains a public subnet for hosting a bastion host and a private subnet for hosting App-A. VPC B contains a private subnet for hosting App-B. Both VPCs use different IPv4 CIDR blocks to avoid IP address conflicts.

A VPC peering connection is established between the two VPCs to enable private communication using private IP addresses. Route tables are configured to direct traffic between the VPCs, while security groups control access to the EC2 instances.

The project also demonstrates secure SSH access through a bastion host and connectivity verification using ICMP ping commands. The main objective is to understand how two separate AWS virtual networks can communicate securely without exposing private application instances directly to the internet.

2. PROJECT OBJECTIVES

The main objectives of this project are:

1. To create two separate Amazon Virtual Private Clouds.
2. To configure different IPv4 CIDR blocks for both VPCs.
3. To create public and private subnets.
4. To configure an Internet Gateway for internet access in VPC A.
5. To launch EC2 instances in the required subnets.
6. To configure security groups for secure SSH and ICMP communication.
7. To establish a VPC Peering connection between VPC A and VPC B.
8. To configure route tables for communication between both VPCs.
9. To verify connectivity using ping and SSH commands.
10. To understand secure communication between private cloud networks.

3. AWS SERVICES AND TECHNOLOGIES USED

3.1 Amazon VPC

Amazon Virtual Private Cloud (VPC) allows users to create an isolated virtual network within AWS. In this project, two VPCs are created to demonstrate private communication between separate networks.

3.2 Subnets

Subnets divide a VPC into smaller network segments. A public subnet is used for the bastion host, while private subnets are used for application instances.

3.3 Internet Gateway

An Internet Gateway allows communication between a VPC and the internet when the appropriate route table and security rules are configured.

3.4 Route Tables

Route tables determine where network traffic is directed. Routes are configured to allow communication between VPC A and VPC B through the peering connection.

3.5 Security Groups

Security groups act as virtual firewalls for EC2 instances. They control incoming and outgoing network traffic based on defined rules.

3.6 Amazon EC2

Amazon Elastic Compute Cloud (EC2) provides virtual servers in AWS. Three EC2 instances are used in this project: one bastion host and two private application instances.

3.7 SSH and Key Pairs

SSH is used to securely connect to Linux EC2 instances. The EC2 key pair provides authentication for remote access.

3.8 VPC Peering

VPC Peering creates a private networking connection between two VPCs, allowing resources to communicate using private IP addresses.

4. PROJECT ARCHITECTURE

The project consists of two separate VPCs connected through a VPC Peering connection.

VPC A uses the CIDR block `10.10.0.0/16` and contains:

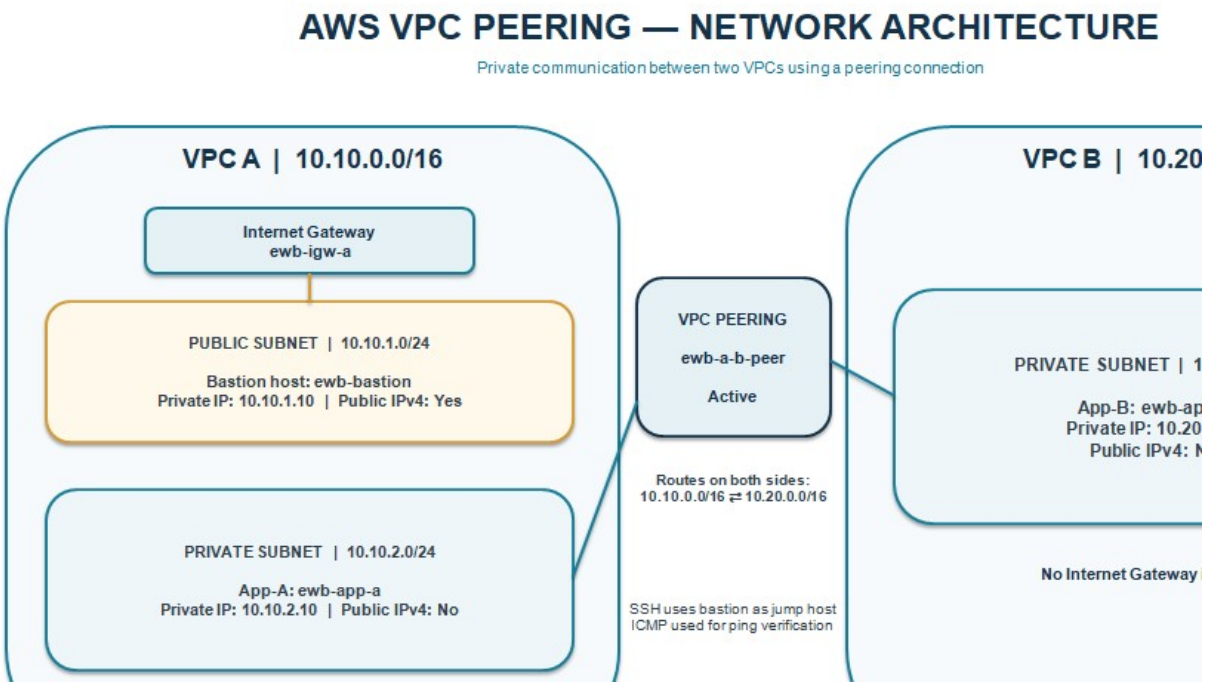
- Public subnet: `10.10.1.0/24`
- Private subnet: `10.10.2.0/24`
- Bastion host: `10.10.1.10`
- App-A: `10.10.2.10`

VPC B uses the CIDR block `10.20.0.0/16` and contains:

- Private subnet: `10.20.1.0/24`
- App-B: `10.20.1.10`

An Internet Gateway is attached to VPC A. The bastion host acts as a secure entry point for accessing the private EC2 instances.

The VPC Peering connection allows App-A and App-B to communicate privately using their private IP addresses.



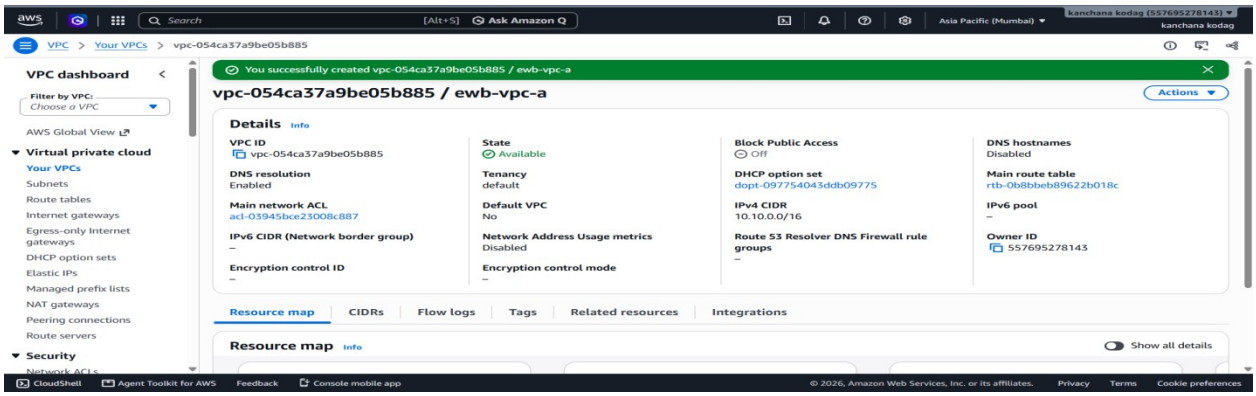
5. Network Design and IP Address Plan

Resource	Name	IPv4 CIDR / Address	Role
VPC A	ewb-vpc-a	10.10.0.0/16	First
Public subnet A	ewb-a-public	10.10.1.0/24	Basti
Private subnet A	ewb-a-private	10.10.2.0/24	App-
VPC B	ewb-vpc-b	10.20.0.0/16	Seco
Private subnet B	ewb-b-private	10.20.1.0/24	App-
Bastion	ewb-bastion	10.10.1.10	Publi host

6. IMPLEMENTATION AND CONFIGURATION

6.1 VPC A created

This screenshot shows the successful creation of VPC A (`ewb-vpc-a`) with the IPv4 CIDR block `10.10.0.0/16`. This VPC acts as the primary network and contains the public subnet for the bastion host and a private subnet for App-A.



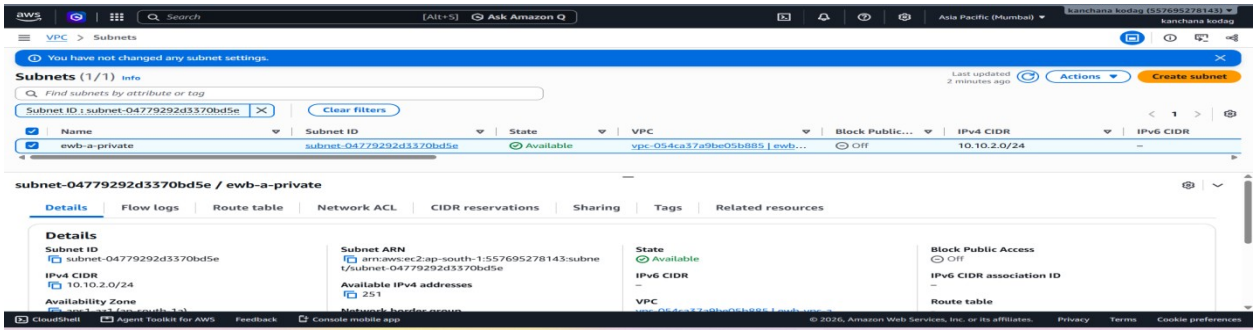
01-vpc-a.png

6.2 VPC A subnets

This screenshot shows the two subnets created inside VPC A:

- Public subnet: `10.10.1.0/24`
- Private subnet: `10.10.2.0/24`

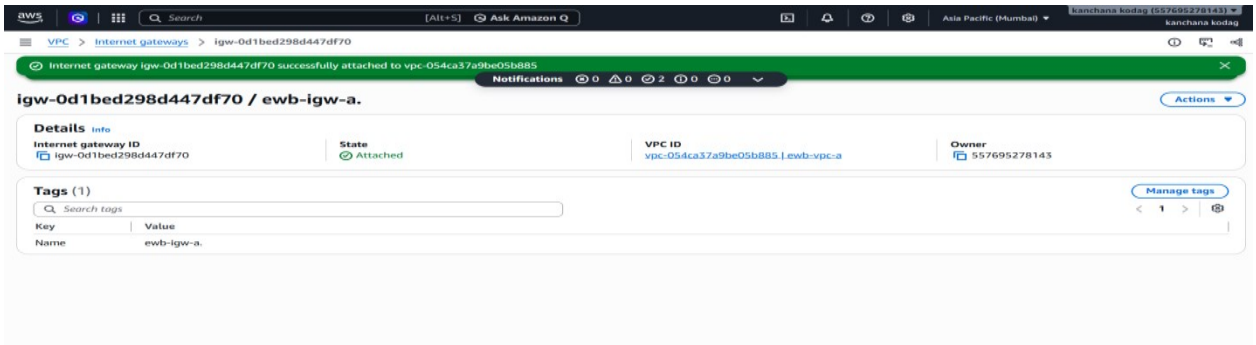
The public subnet is used for the bastion host, while the private subnet is used for the App-A instance.



02-vpc-a-subnets.png

6.3 Internet Gateway attached

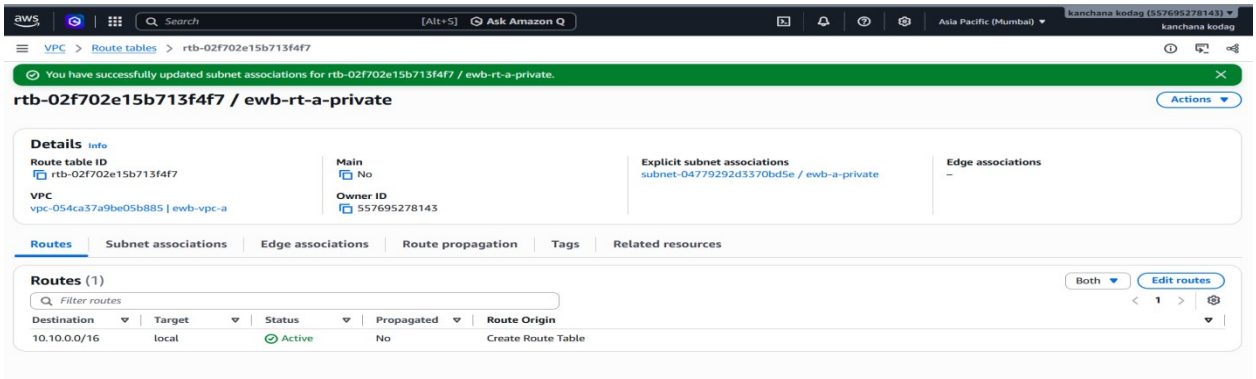
This screenshot shows the Internet Gateway named `ewb-igw-a` attached to VPC A. It provides internet connectivity to resources in the public subnet when the appropriate route and security rules are configured.



03-internet-gateway.png

6.4 VPC A public route table

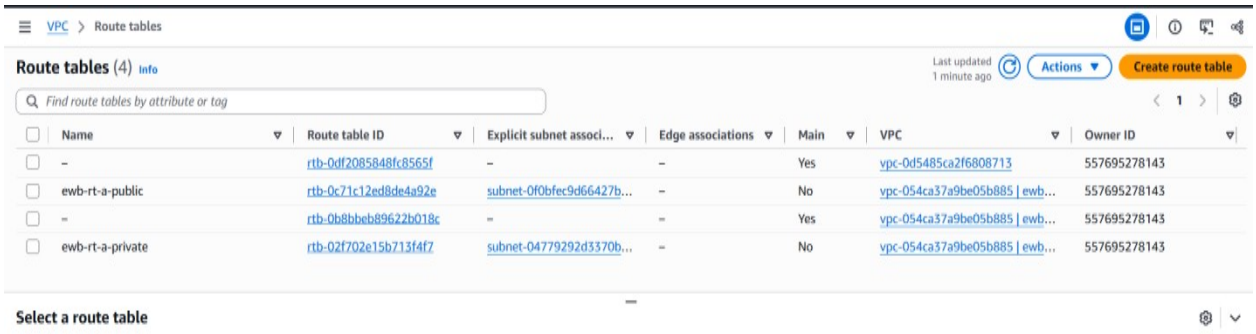
This screenshot displays the public route table `ewb-rt-a-public`. It contains the local route for VPC A and the default route `0.0.0.0/0` pointing to the Internet Gateway. This allows the bastion host to communicate with the internet.



04-public-route.png

6.5 VPC A private route table

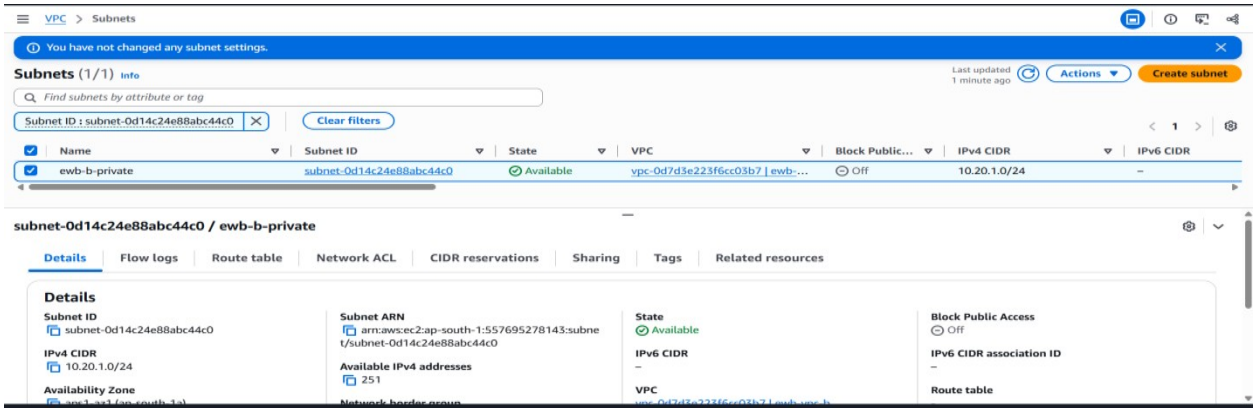
This screenshot shows the private route table `ewb-rt-a-private`, associated with the private subnet of VPC A. Initially, it contains the local route `10.10.0.0/16`, allowing communication within VPC A.



05-private-route.png

6.6 VPC B and private subnet

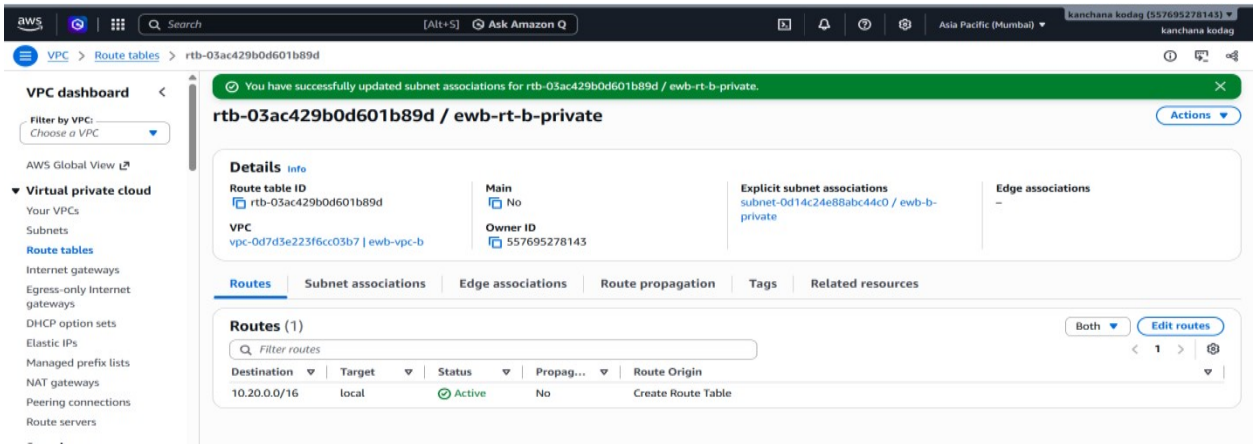
This screenshot shows the creation of VPC B (`ewb-vpc-b`) with CIDR block `10.20.0.0/16` and its private subnet `10.20.1.0/24`. This VPC hosts App-B. It does not require an Internet Gateway for this private communication project.



06-vpc-b-subnet.png

6.7 VPC B route table

This screenshot displays the route table `ewb-rt-b-private`, associated with the private subnet in VPC B. It initially contains the local route `10.20.0.0/16`, which allows communication within VPC B.

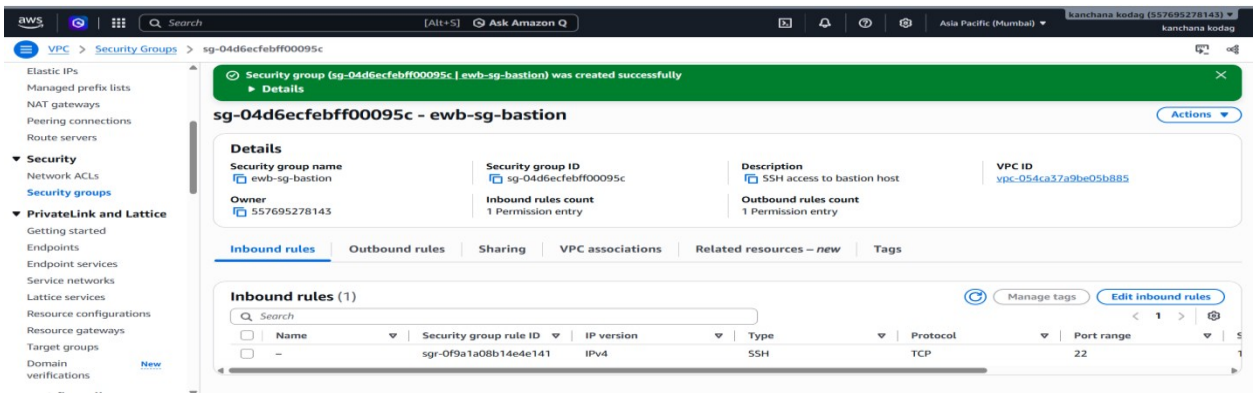


07-vpc-b-route.png

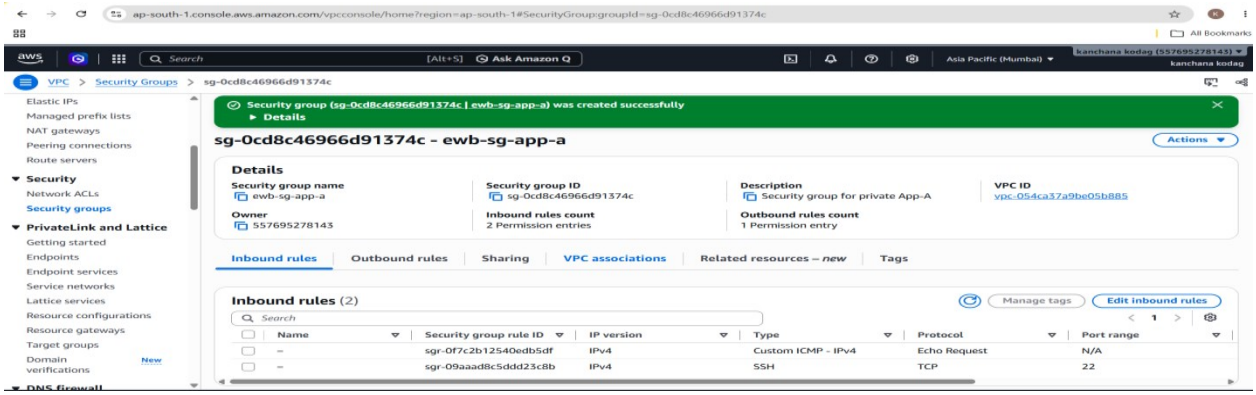
6.8 Security group rules

This screenshot shows the inbound security rules configured for the three security groups:

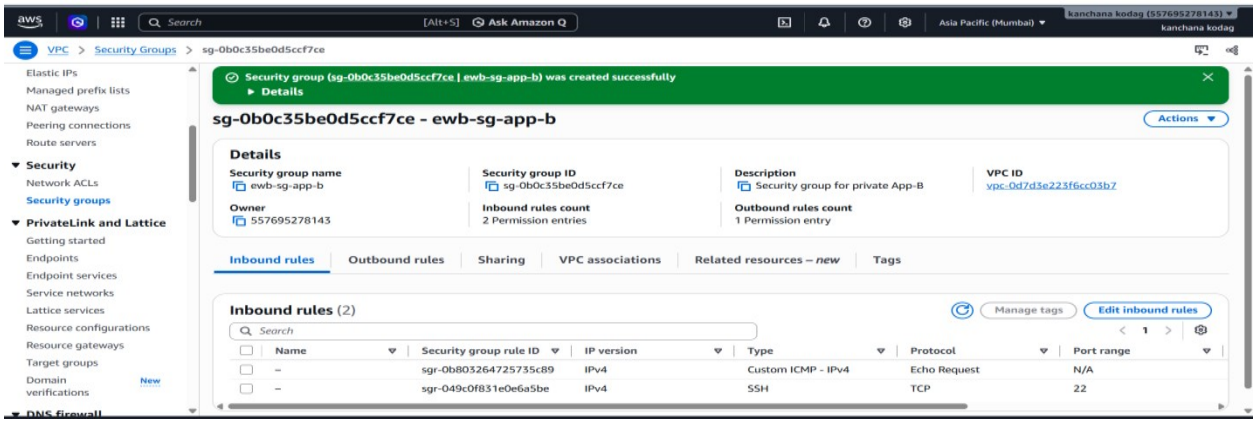
- `ewb-sg-bastion`: Allows SSH (TCP port 22) from the user's public IP address.



`ewb-sg-app-a`: Allows SSH from the bastion host and ICMP Echo Request from App-B.



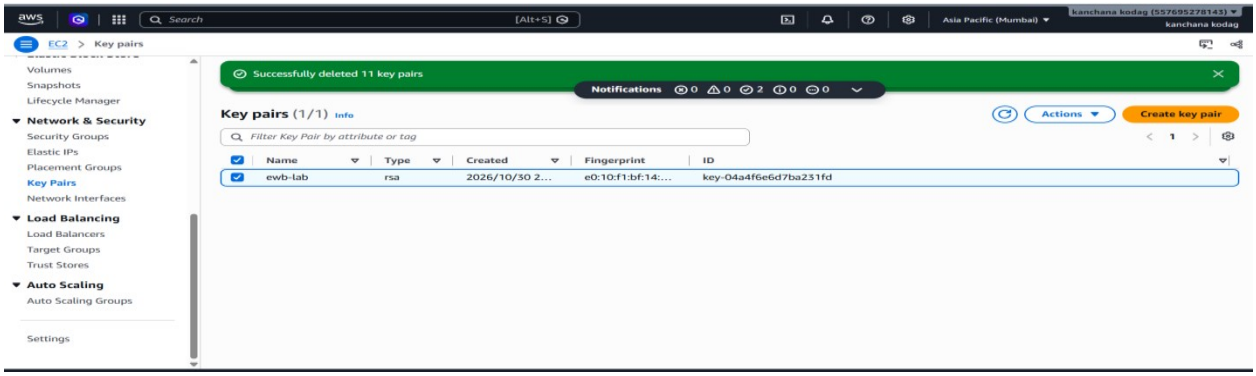
`ewb-sg-app-b`: Allows SSH from the bastion host and ICMP Echo Request from App-A.



08-security-groups.png

6.9 Key pair

This screenshot shows the creation of the EC2 key pair named `ewb-lab`. The key pair is used for secure SSH authentication when connecting to the EC2 instances. Only the key pair name should be visible; never show or upload the private PEM key contents.



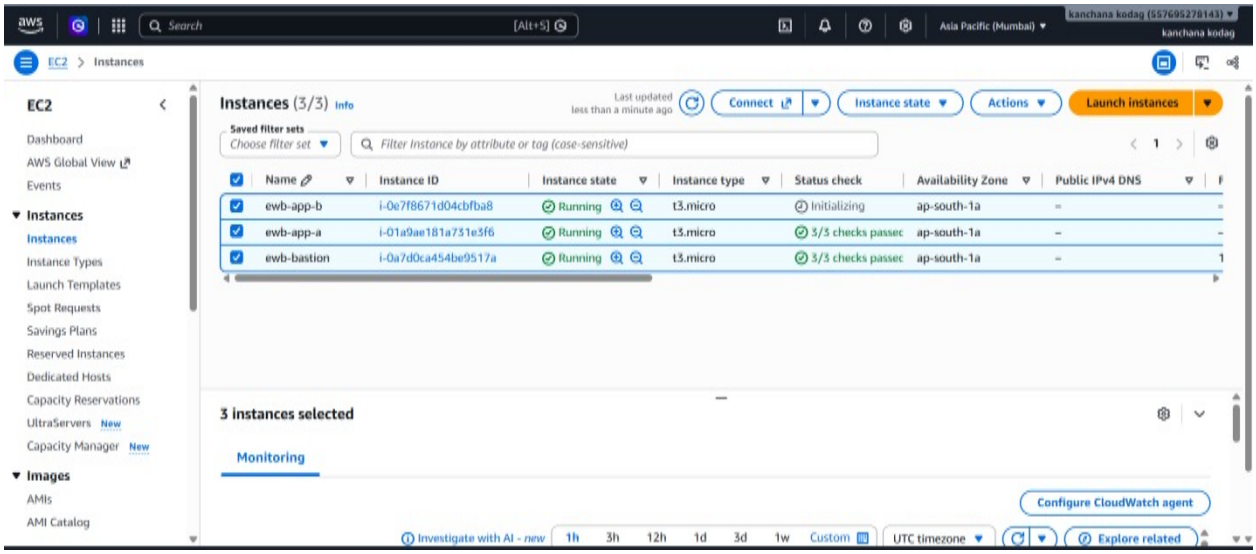
09-key-pair.png

6.10 EC2 instances

This screenshot shows the three EC2 instances launched using Amazon Linux 2023:

- `ewb-bastion` — public subnet of VPC A.
- `ewb-app-a` — private subnet of VPC A.
- `ewb-app-b` — private subnet of VPC B.

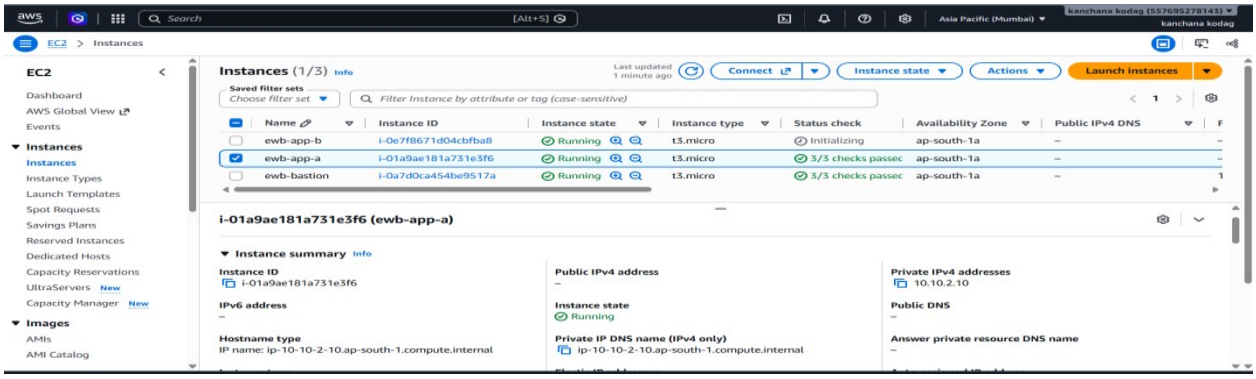
The instances provide the environment required to test secure access and communication between the two VPCs



10-ec2-instances.png

6.11 Private and public IP verification

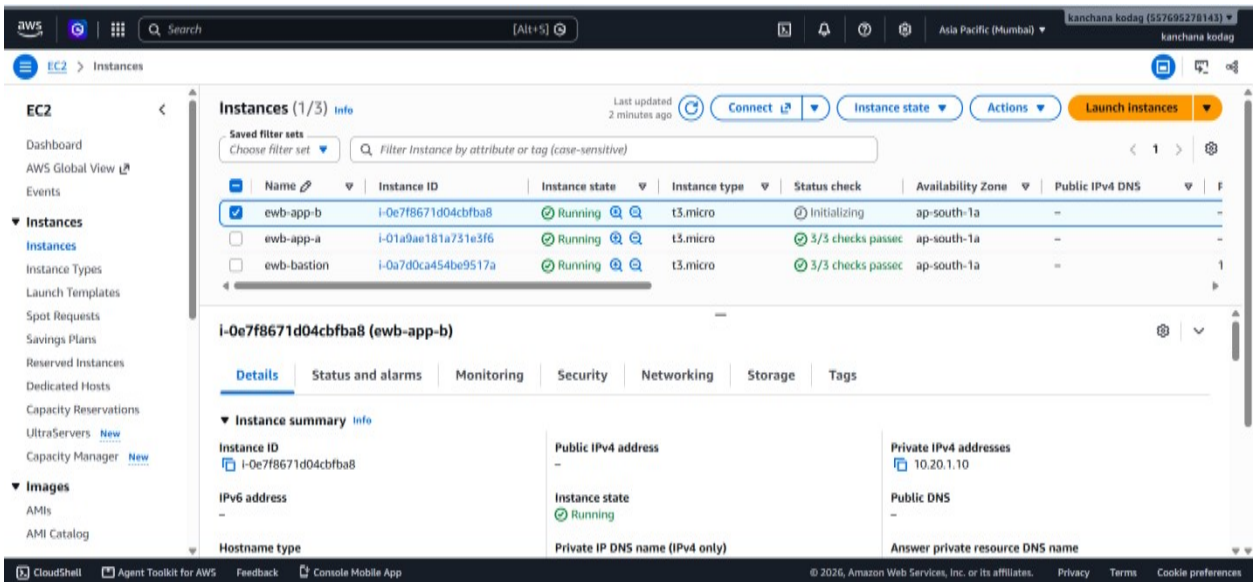
This screenshot verifies the IP address configuration of all three EC2 instances. The bastion host has a public IPv4 address, while App-A (`10.10.2.10`) and App-B (`10.20.1.10`) use private IPv4 addresses and do not have public IPv4 addresses. This demonstrates that the application instances remain private.



11-private-public-ips.png

6.12 Bastion SSH login

This screenshot shows a successful SSH connection from the local computer to the bastion host using the `ewb-lab` key pair. The bastion host acts as a secure entry point for accessing the private EC2 instances.



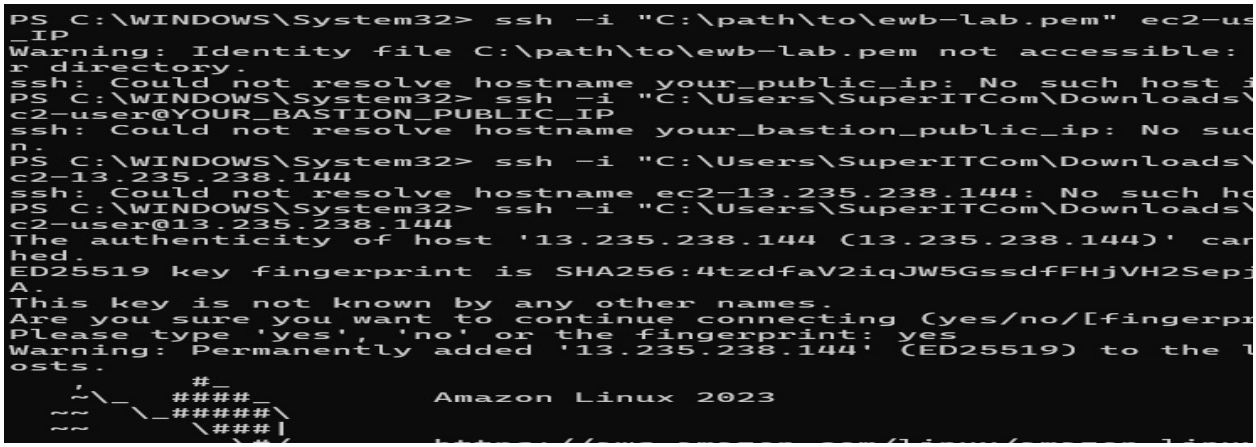
12-bastion-ssh.png

6.13 Baseline ping before peering

This screenshot shows the initial connectivity test from App-A to App-B using:

```
ping -c 4 10.20.1.10
```

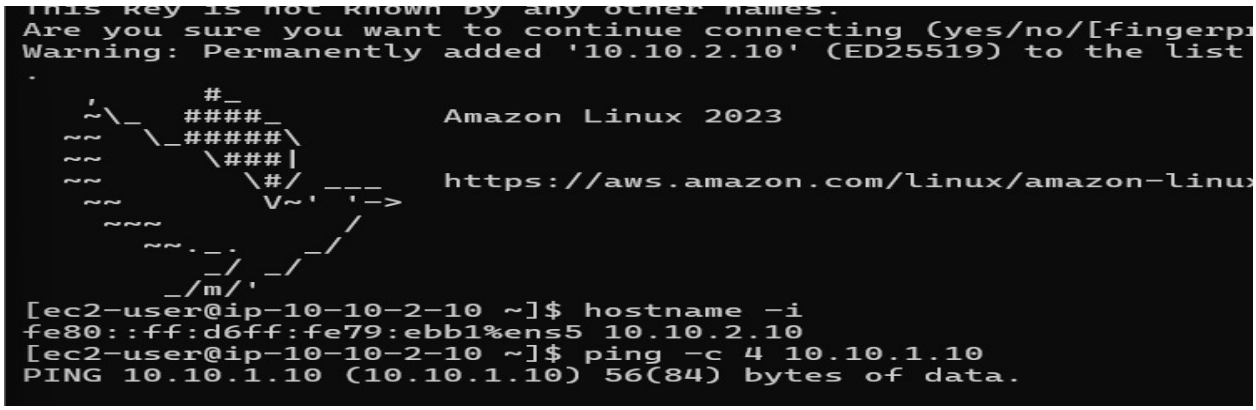
Before configuring VPC Peering and the required routes, App-A is expected to receive no replies from App-B. This establishes the baseline result for comparison with the later connectivity test.



13-baseline-ping.png

6.14 Active VPC peering connection

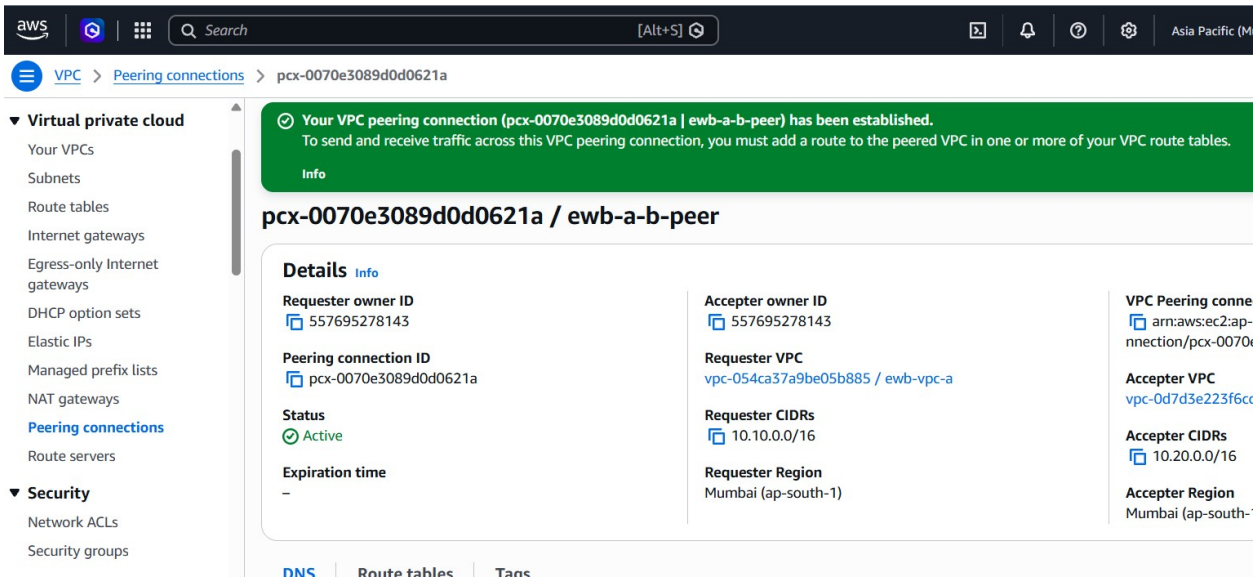
This screenshot shows the VPC peering connection named `ewb-a-b-peer`, with VPC A as the requester and VPC B as the accepter. The status should display Active, confirming that the peering request has been accepted and the connection is established.



14-peering-active.png

6.15 VPC A private peering route

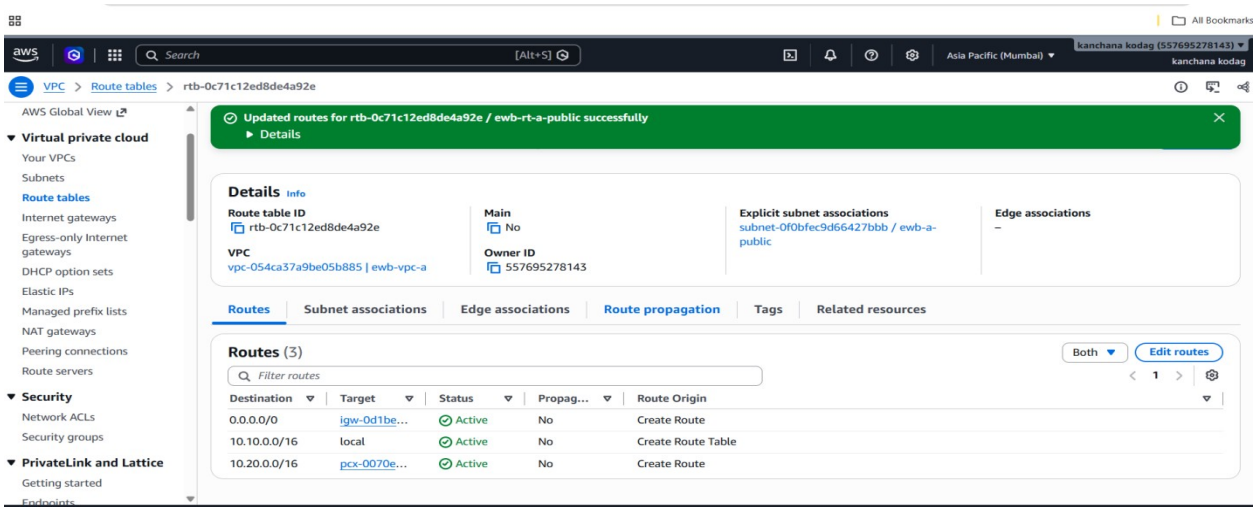
This screenshot shows the updated private route table of VPC A. A route is added with destination `10.20.0.0/16` and target set to the VPC peering connection. This allows App-A to send traffic to the network range of VPC B.



15-route-a-private.png

6.16 VPC B private peering route

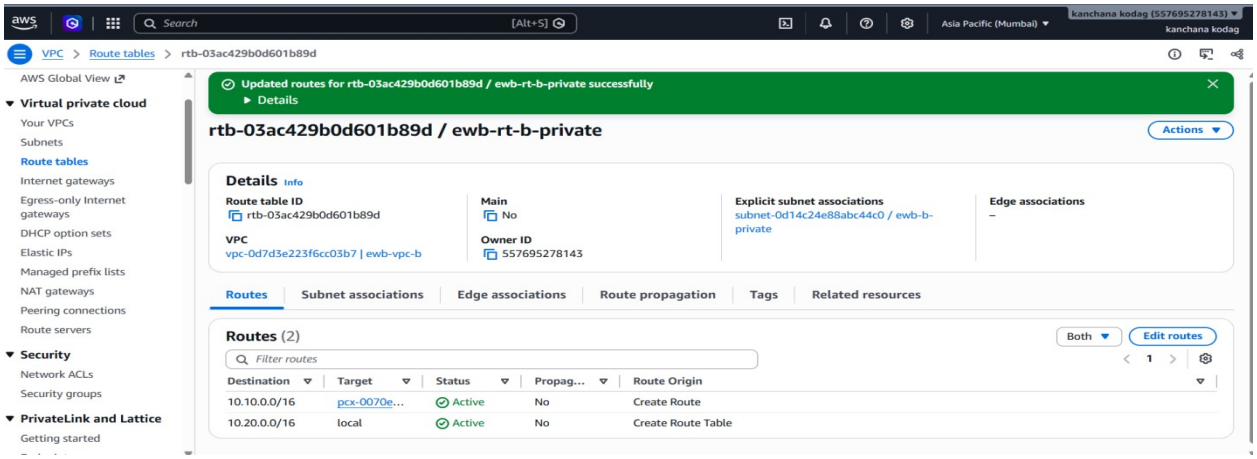
This screenshot shows the updated private route table of VPC B. A route is added with destination `10.10.0.0/16` and target set to the VPC peering connection. This provides the return path for traffic from VPC B to VPC A.



16-route-b-private.png

6.17 VPC A public peering route

This screenshot shows the updated public route table of VPC A. It contains the local route, the default internet route through `ewb-igw-a`, and the route `10.20.0.0/16` through the VPC peering connection. This route allows the bastion host to reach App-B through the peering connection when using SSH through the bastion.



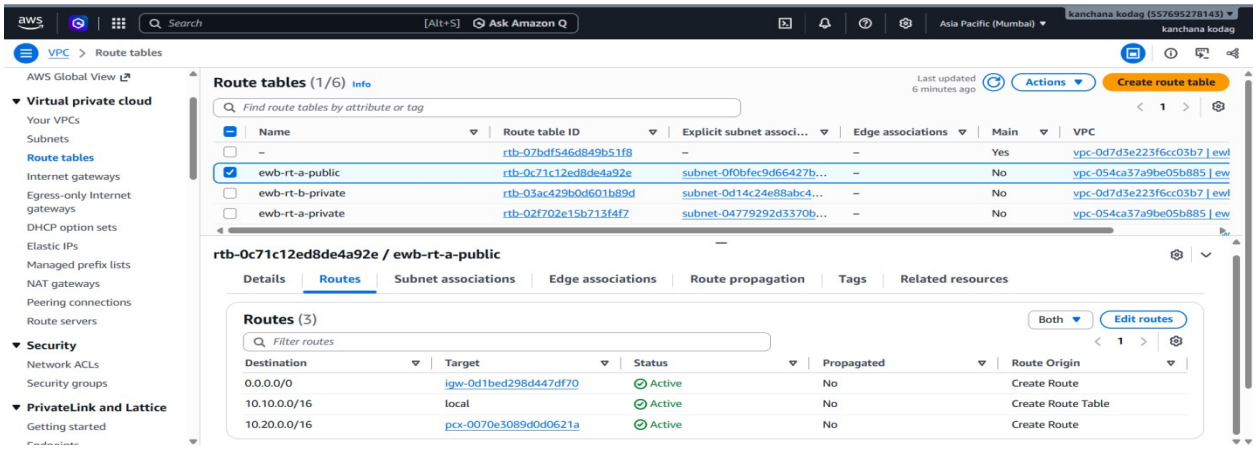
17-route-a-public.png

6.18 App-A to App-B ping

This screenshot shows the successful connectivity test from App-A to App-B using:

```
ping -c 4 10.20.1.10
```

Successful replies confirm that traffic can travel from VPC A to VPC B through the peering connection using private IP addresses.



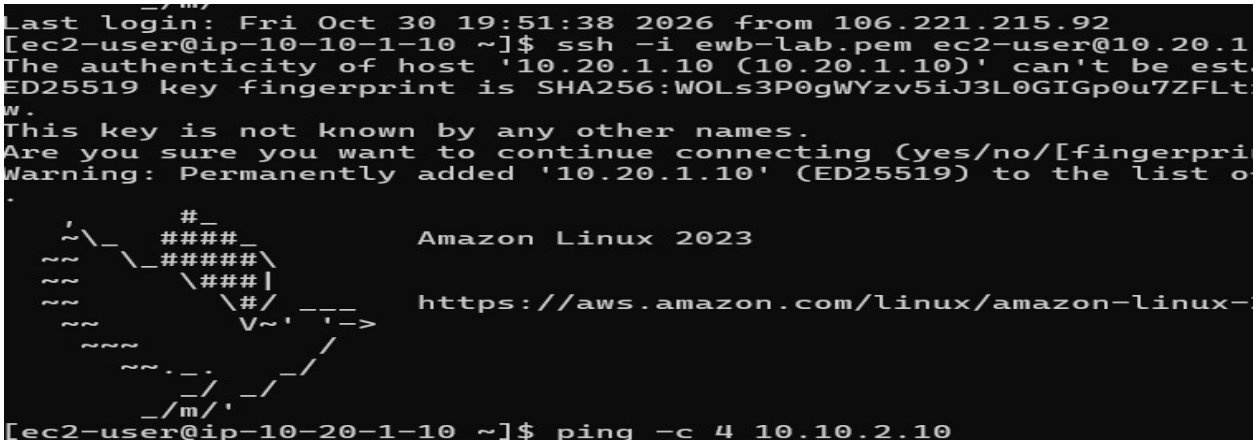
18-ping-a-to-b.png

6.19 App-B to App-A ping

This screenshot shows the reverse connectivity test from App-B to App-A using:

```
ping -c 4 10.10.2.10
```

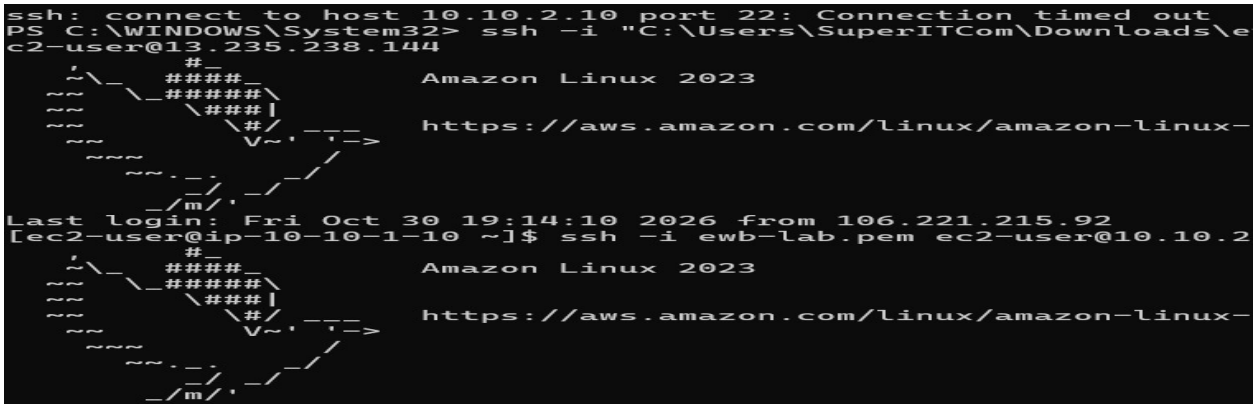
Successful replies confirm that communication is working in the reverse direction as well. This verifies that routing and security rules are configured for bidirectional communication.



19-ping-b-to-a.png

6.20 SSH to App-B through bastion

This screenshot shows a successful SSH connection to App-B through the bastion host using the SSH jump-host method. App-B remains in a private subnet without a public IPv4 address. This demonstrates secure remote access to a private EC2 instance through the bastion host.



20-app-b-ssh.png

6.21 ICMP failure test

This screenshot shows the controlled failure test. The ICMP Echo Request inbound rule for App-B, with source `10.10.2.10/32`, is temporarily removed. When the ping command is repeated from App-A, the expected result is no replies. This demonstrates how security group rules control network communication.



21-icmp-failure.png

6.22 ICMP recovery test

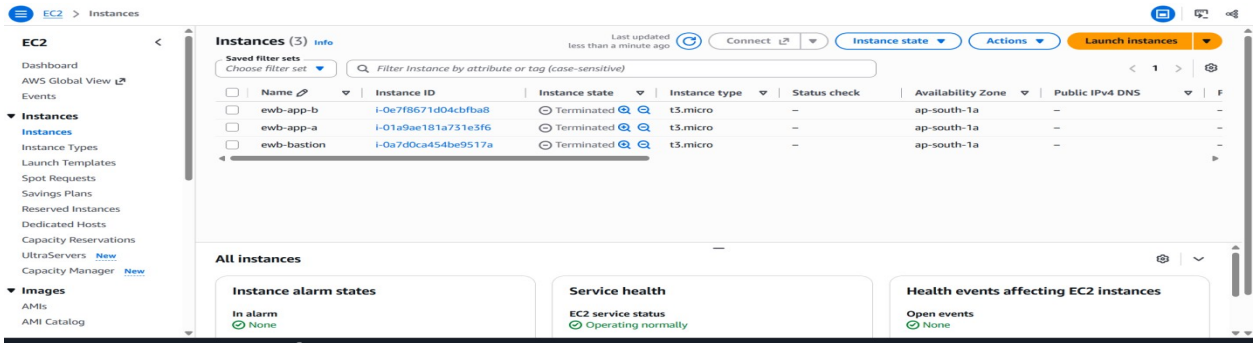
This screenshot shows the successful recovery test after restoring the ICMP Echo Request inbound rule in `ewb-sg-app-b`. The ping command is executed again from App-A to App-B, and replies should resume. This confirms that restoring the required security rule restores network connectivity



22-icmp-restored.png

6.23 Cleanup evidence

This screenshot shows the cleanup of AWS resources after completing the project, if cleanup was required. It may include terminated EC2 instances, deleted VPC peering connections, removed route entries, deleted security groups and subnets, and deleted VPCs and the Internet Gateway. It demonstrates that unused lab resources were removed to avoid unnecessary AWS charges.



23-cleanup.png

7. TEST RESULTS SUMMARY

Test Description	Expected Result	Actual Result
Bastion SSH	Successful login	SSH login succes to Ubuntu 22.04 and bash prompt.
Baseline ping	No replies before peering	No replies receive 10.0.2.148 (Requ ping failed).
App-A to App-B	Replies after peering	Successful replies 10.0.2.148 (4 pac received, 0% loss)
App-B to App-A	Replies after peering	Successful replies (3 packets sent, 3 loss).
SSH to App-B via bastion	Successful private access	Private key accep session establishe private IP 10.0.2.
ICMP rule removed	Ping fails	Ping from App-A

8. TROUBLESHOOTING

Issue	Checks to perform
Bastion SSH timeout	Check bastion public IP, administr its security group, public subnet a fault route to attached IGW.
App-A SSH failure	Check App-A private IP, TCP 22 ru 10.10.1.10/32, key pair and ec2-u
App-B SSH failure	Check VPC A public route to 10.2 turn route and App-B SSH rule fro
Ping failure after peering	Check Active peering status, route

9. CONCLUSION

The AWS VPC Peering project demonstrates how two separate Amazon Virtual Private Clouds can communicate privately using a VPC Peering connection.

The project covers the creation of VPCs, subnet configuration, Internet Gateway setup, route table configuration, EC2 deployment and security group management.

The implementation also explains the importance of configuring routes in both directions and allowing the required network traffic through security groups.

Through this project, practical knowledge of AWS networking, private IP communication, bastion host access, SSH connectivity and network troubleshooting is developed.

10. LEARNING OUTCOMES

After completing this project, the following concepts were understood:

1. Creation and configuration of Amazon VPCs.
2. Understanding IPv4 CIDR blocks and subnetting.
3. Difference between public and private subnets.
4. Configuration of Internet Gateways.
5. Configuration of route tables.
6. Creation and management of EC2 instances.
7. Configuration of security groups.
8. Establishing VPC Peering connections.
9. Secure access to private EC2 instances using bastion hosts.
10. Testing network connectivity using SSH and ICMP ping commands.
11. Understanding the importance of network security and routing.

11.Appendix A — Screenshot Checklist

No.	Evidence	Suggested fil
1	VPC A created	01-vpc-a.png
2	VPC A subnets	02-vpc-a-subr
3	Internet Gateway attached	03-internet-ga
4	VPC A public route table	04-public-rou
5	VPC A private route table	05-private-rou
6	VPC B and private subnet	06-vpc-b-subr
7	VPC B private route table	07-vpc-b-rout
8	Security group inbound rules	08-security-gr
9	Key pair name	09-key-pair.pr
10	Three EC2 instances	10-ec2-instan
11	Private/public IP verification	11-private-pu
12	Bastion SSH login	12-bastion-ssl
13	Baseline ping	13-baseline-pi
14	Active peering	14-peering-ac
15	VPC A private route	15-route-a-pr
16	VPC B private route	16-route-b-pr
17	VPC A public route	17-route-a-pu
18	App-A to App-B ping	18-ping-a-to-b
19	App-B to App-A ping	19-ping-b-to-a